

PROJETO HIDEX - APLICATIVO MAYA YAMAMOTO**Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas****INTEGRANTES DO PROJETO e RA'S**

Bianca Ferreira Morais	-	25027615
Enzo Ribeiro da Silva	-	25027384
Rikelmy Anacleto Lemos	-	25027355
Victor Melo	-	24026801
Matheus Canelutti Lacerda	-	26029073

São Paulo

2026

Sumário

1. Qualidade de Software	2
1.1. Modelo que qualidade de software (Diagrama/Design)	2
1.2. Processo (plano) de gerenciamento de qualidade de software	2
1.3. Identificação de atributos de qualidade da norma 25010.	2
1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.	2
2. Teste de Software	6
2.1. Plano de Teste	6
2.2. Apresentar 2 testes unitários	6
2.3. Apresentar 2 testes de integração	6
2.4. Apresentar um teste de usuário (SISTEMA)	7
3.0. Referências Bibliográficas	7

1. Qualidade de Software

A qualidade de software é definida como o grau em que um sistema satisfaz as necessidades declaradas e implícitas de seus diversos usuários e partes interessadas, entregando valor, segurança e eficiência. Em um projeto de engenharia de software, a qualidade não é um aspecto subjetivo, mas sim um conjunto de características mensuráveis e controláveis. Ela é essencial pois garante que o produto final seja resiliente a falhas, proteja informações sensíveis, apresenta tempos de resposta adequados e possua uma estrutura que permita evoluções futuras sem a introdução de erros críticos.

Fundamentação pela Norma ISO/IEC 25010

A base conceitual adotada para este projeto é a norma **ISO/IEC 25010**, que substituiu a ISO 9126 e estabelece um modelo de qualidade de produto composto por oito características principais. A qualidade é avaliada de forma holística em duas dimensões:

1. **Qualidade Funcional (Requisitos Funcionais):** Refere-se à capacidade do software de prover funções que atendam às necessidades especificadas quando usado sob condições estabelecidas. É a verificação se o sistema "faz o que deveria fazer".
2. **Qualidade Estrutural (Requisitos Não Funcionais):** Refere-se a como o sistema se comporta. Envolve aspectos críticos como:
 - **Desempenho:** Eficiência no uso de recursos e tempo de resposta.
 - **Segurança:** Proteção de dados e resistência a acessos não autorizados.
 - **Usabilidade:** Facilidade de aprendizado e satisfação do usuário ao interagir com a interface.
 - **Confiabilidade:** Capacidade do sistema de permanecer operacional e recuperar-se de erros.
 - **Manutenibilidade:** Facilidade com que o código pode ser modificado para correções ou melhorias.

Medição e Garantia da Qualidade

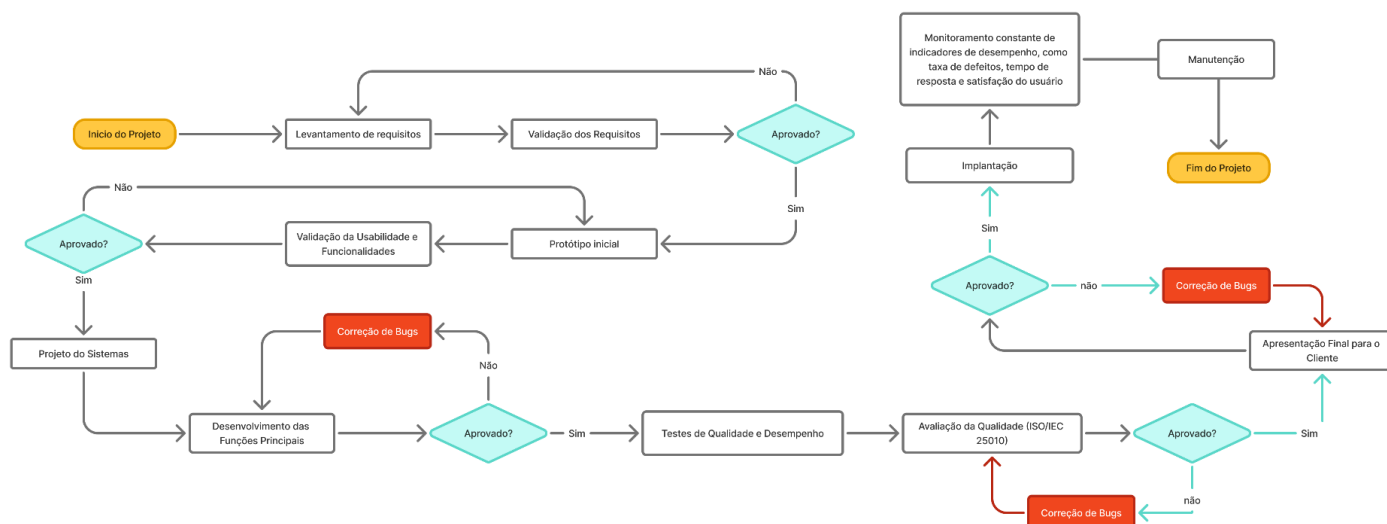
Para assegurar que o software atinja os níveis de excelência desejados, a equipe utiliza uma estratégia baseada em processos de **Garantia da Qualidade (QA)** e **Controle da Qualidade (QC)**:

- **Medição:** A qualidade será quantificada através de métricas específicas, como a densidade de defeitos, o tempo médio para reparo (MTTR), a cobertura de testes unitários e o tempo de resposta das principais transações do sistema.
- **Garantia:** A equipe implementa padrões de codificação, revisões por pares (*code reviews*) e análise estática de código para prevenir a ocorrência de falhas desde as etapas iniciais do desenvolvimento.

Validação: O controle será realizado através de ciclos de testes estruturados (unitários, integração e sistema), garantindo que tanto os requisitos funcionais quanto os não funcionais sejam atendidos de forma eficiente, segura e conforme as especificações do projeto.

1.1. Modelo que qualidade de software (Diagrama/Design)

O modelo de qualidade adotado para este projeto fundamenta-se na norma **ISO/IEC 25010**, que descreve as características essenciais para um produto de software de alta performance. O diagrama abaixo detalha como os atributos de qualidade são distribuídos no design da solução, servindo como guia para a arquitetura do sistema e para a definição dos critérios de aceitação.



<https://www.figma.com/board/NF4JpUkluvVoFm2SHar4uR/Untitled?node-id=0-1&p=f&t=236yU66cUgHxdkp-0>

O processo inicia com o Levantamento de Requisitos e sua respectiva Validação. Após a primeira aprovação, o fluxo avança para a criação do Protótipo Inicial, que é submetido à Validação da Usabilidade e Funcionalidades. Caso o protótipo não seja aprovado, o fluxo retorna ao início para o refinamento dos requisitos; se aprovado, segue-se para o Projeto do Sistemas, onde a arquitetura é definida.

Com a arquitetura estabelecida, inicia-se o Desenvolvimento das Funções Principais. Esta fase possui um ciclo de correção interna: se o código não for aprovado, passa por Correção de Bugs antes de prosseguir. Uma vez validado, o software entra na fase de Testes de Qualidade e Desempenho, seguida pela rigorosa Avaliação da Qualidade (ISO/IEC 25010).

A avaliação baseada na ISO/IEC 25010 garante que o produto atende a critérios como confiabilidade, eficiência e segurança. Se houver falhas, o projeto retorna para a Correção de Bugs e reavaliação. Após a aprovação técnica, ocorre a Apresentação Final para o Cliente. Se o cliente solicitar ajustes (Não Aprovado), o fluxo retorna novamente para a correção; se aprovado (Sim), o sistema segue para a Implantação.

Após a Implantação, o sistema entra em um estado de Monitoramento constante de indicadores, como taxa de defeitos e satisfação do usuário. Esse monitoramento alimenta a fase de Manutenção,

garantindo a longevidade e estabilidade do software. O processo é concluído formalmente no nó Fim do Projeto.

1.2. Processo de gerenciamento de qualidade de software

O processo de gerenciamento da qualidade de software consiste no conjunto de atividades e práticas voltadas a garantir que o produto desenvolvido atenda aos requisitos especificados, às necessidades dos usuários e aos padrões estabelecidos. Esse processo é essencial para assegurar a confiabilidade, eficiência, manutenibilidade e usabilidade do sistema.

Inicialmente, realiza-se o **planejamento da qualidade**, etapa na qual são definidos os padrões, métricas, critérios de aceitação e métodos que serão utilizados para avaliar o software. Nessa fase, são estabelecidas diretrizes baseadas em boas práticas de engenharia de software, normas técnicas e objetivos do projeto.

Em seguida, ocorre a **garantia da qualidade**, que envolve a aplicação sistemática de processos e metodologias durante o desenvolvimento. Essa etapa tem como objetivo prevenir defeitos por meio da padronização de processos, revisões técnicas, auditorias e acompanhamento contínuo das atividades realizadas pela equipe.

Posteriormente, é realizado o **controle da qualidade**, que consiste na verificação e validação do produto por meio de testes e inspeções. Entre as principais práticas estão os testes unitários, testes de integração, testes de sistema e testes de aceitação, que permitem identificar falhas e assegurar que o software esteja em conformidade com os requisitos definidos.

Além disso, o processo de qualidade é contínuo e iterativo, envolvendo o monitoramento constante de indicadores de desempenho, como taxa de defeitos, tempo de resposta e satisfação do usuário. Com base nesses dados, são realizadas ações corretivas e melhorias contínuas no sistema.

Dessa forma, o gerenciamento da qualidade de software contribui significativamente para a entrega de um produto confiável, alinhado às expectativas do cliente e com maior valor agregado, reduzindo riscos e custos relacionados a falhas e retrabalho.

1.3. Identificação de atributos de qualidade da norma 25010.

A norma **ISO/IEC 25010** estabelece oito características de qualidade, porém, para garantir a eficiência do projeto e o foco da equipe de desenvolvimento, foram selecionados os cinco atributos de maior relevância para o sistema proposto. Cada atributo abaixo é contextualizado com base em sua importância estratégica para a solução final:

1. Adequação Funcional (Funcionalidade)

Este é o atributo primário do projeto. Ele garante que o software forneça todas as funções necessárias para atender aos requisitos levantados com o cliente. É essencial para assegurar que os cálculos de dados, os fluxos de cadastro e os processos de negócio operem com completitude e correção funcional, entregando exatamente o que foi proposto no escopo.

2. Segurança

Trata-se de um atributo crítico, dada a natureza dos dados processados pelo sistema (informações de usuários e possíveis transações). A segurança garante a confidencialidade, integridade e autenticidade das informações, prevenindo acessos não autorizados e garantindo a conformidade com as diretrizes da LGPD (Lei Geral de Proteção de Dados).

3. Usabilidade

A usabilidade foca na facilidade de aprendizado e na satisfação do usuário ao interagir com o sistema. É vital para o sucesso do projeto, pois um design intuitivo e uma interface operável reduzem a curva de aprendizado e minimizam erros operacionais cometidos pelos usuários finais, aumentando a eficiência do uso diário.

4. Confiabilidade

A confiabilidade mede a capacidade do sistema de permanecer operacional e executar suas funções sob condições estabelecidas por um período de tempo. É fundamental para garantir que o sistema seja resiliente a falhas e possua mecanismos de recuperação de dados, assegurando que informações críticas não sejam perdidas em caso de instabilidades técnicas.

5. Manutenibilidade

Este atributo refere-se à facilidade com que o software pode ser modificado pela equipe técnica. No contexto do projeto, a manutenibilidade é essencial para permitir correções rápidas de bugs e a implementação de melhorias futuras sem comprometer a estabilidade do sistema já existente, garantindo a longevidade do código.

Relatório de Utilização da Norma 25010 no Desenvolvimento

A utilização da norma ISO/IEC 25010 não ocorre de forma isolada, mas sim como um guia transversal em todo o ciclo de vida de desenvolvimento:

Engenharia de Requisitos: Os atributos da norma são usados como *checklists* para identificar requisitos não funcionais que, de outra forma, poderiam ser negligenciados, tais como: adequação funcional,

eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade.

Design e Arquitetura: As decisões de design são tomadas com o objetivo de suportar os atributos de qualidade definidos, como a adoção de modularização para facilitar a manutenibilidade, implementação de mecanismos de segurança como criptografia e definição de uma arquitetura escalável para garantir melhor desempenho e confiabilidade do sistema.

Execução de Testes: Os planos de teste são estruturados para validar não apenas o "o quê" o sistema faz, mas "como" ele se comporta sob estresse, garantindo que os níveis de qualidade definidos pela norma sejam atingidos antes da entrega final.

1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

A utilização da norma ISO/IEC 25010 neste projeto transcende a mera citação teórica, funcionando como o alicerce para a definição de **Requisitos Não Funcionais (RNFs)** e métricas de aceitação. A norma foi incorporada ao ciclo de vida de desenvolvimento (SDLC) como um guia de referência para garantir que o produto final seja robusto, seguro e eficiente.

Incorporação aos Requisitos e Design

Desde a fase de levantamento, a norma foi utilizada para extrair necessidades que não aparecem em fluxos de negócio comuns. Atributos como **Segurança** e **Manutenibilidade** foram traduzidos em decisões de design, como a escolha de uma arquitetura modular que facilita a manutenção e a implementação de protocolos de criptografia para proteção de dados sensíveis. Sem a referência da norma, tais aspectos poderiam ser negligenciados em favor apenas das funcionalidades visíveis.

Definição de Métricas e Critérios de Qualidade

Para que a qualidade fosse mensurável, estabelecemos critérios de aceitação baseados nas sub-características da 25010:

- **Métrica de Desempenho:** O tempo de resposta das transações principais deve ser inferior a 2 segundos em 95% das requisições.
- **Métrica de Usabilidade:** O índice de completitude de tarefas por novos usuários deve ser superior a 90% sem auxílio externo.
- **Métrica de Manutenibilidade:** O código deve manter uma cobertura de testes unitários de, no mínimo, 70% nas classes de lógica de negócio.

Verificação através de Testes e Validações

A verificação da conformidade com a norma ocorre de forma contínua através do fluxo de aprovação estabelecido:

1. **Validação de Interface:** Testes de usabilidade e prototipagem garantem que os atributos de operabilidade e estética sejam atendidos antes da codificação.
2. **Testes de Estresse e Carga:** Validam a **Confiabilidade** e a **Eficiência de Desempenho**, submetendo o sistema a volumes de dados que simulam o uso real e extremo.
3. **Auditoria de Código e Segurança:** Revisões por pares e ferramentas de análise estática garantem que as diretrizes de **Segurança** e as boas práticas de desenvolvimento (Manutenibilidade) sejam respeitadas em cada *commit*.

A aplicação da ISO/IEC 25010 permitiu à equipe uma visão 360 graus sobre o produto. O resultado é um sistema onde a qualidade é monitorada através de fatos e dados, onde cada "Aprovação" no fluxograma de desenvolvimento significa que o software não apenas "funciona", mas atende aos rigorosos padrões internacionais de engenharia de software, promovendo a melhoria contínua a cada ciclo de entrega.

2. Teste de software

A qualidade de um sistema de software depende diretamente da robustez dos processos de validação aplicados ao longo do seu desenvolvimento. No contexto do App Maya Yamamoto, aplicativo Android voltado ao acompanhamento de exercícios físicos e registro de execuções, a realização de testes estruturados é essencial para garantir que as funcionalidades entreguem ao usuário final uma experiência confiável, segura e intuitiva. focamos nos pilares de **Funcionalidade** e **Confiabilidade**. Ao testar o código em diferentes níveis, garantimos que pequenas falhas de lógica não se tornem erros críticos na mão do usuário final.

A estratégia adotada contempla três níveis complementares de teste:

- **Testes Unitários:** verificam o comportamento isolado de métodos e funções, assegurando que cada unidade de código funcione corretamente de forma independente.
- **Testes de Integração:** validam a comunicação entre os módulos do sistema, garantindo que as diferentes camadas (interface, lógica e persistência) interajam conforme esperado.
- **Testes de Sistema:** avaliam a experiência real de uso, identificando problemas de usabilidade e fluxos que possam comprometer a satisfação do usuário.

O objetivo final é demonstrar que o sistema passou por um processo de validação sistemático antes da entrega, reduzindo riscos de falhas em produção e assegurando aderência aos requisitos funcionais definidos.

2.1. Plano de Teste

O plano de teste define as diretrizes para a validação do sistema:

Tipos de Testes: Unitários, de Integração, de Sistema (Aceitação) e de Carga.

Ferramentas: Foram utilizados JUnit 4 e JUnit 5 integrados ao Android Studio para automação dos testes de lógica, integração e carga. Os testes de interface foram realizados com ferramentas nativas do Android Studio, além de testes manuais de aceitação com usuários.

Responsáveis: Cada integrante da equipe é responsável pelos testes das funcionalidades que desenvolveu, com revisão final realizada pelo líder técnico do projeto.

Momento no Cronograma: Os testes são executados continuamente ao final de cada funcionalidade implementada e antes da geração de cada nova build do sistema.

Critérios de Sucesso: Todos os testes unitários devem apresentar 100% de aprovação, os fluxos críticos de integração devem funcionar corretamente e o usuário deve concluir as tarefas principais sem erros ou auxílio externo.

Critérios de Falha: Qualquer erro de lógica, falha de integração, crash, exceção não tratada ou problema de navegação entre telas é considerado falha no sistema.

Correção de Erros: As falhas são registradas, classificadas por severidade e encaminhadas ao desenvolvedor responsável. Após a correção, os testes são executados novamente até que o sistema seja validado sem erros.

2.2. Apresentar 4 Testes Unitários

Os testes unitários a seguir verificam o comportamento isolado de componentes do sistema, sem dependência de outros módulos, banco de dados ou interface gráfica.

Teste Unitário 01 — Bloqueio de Login com E-mail Vazio

Este teste verifica se a camada de validação da tela de login impede corretamente o avanço quando o campo de e-mail está vazio. Trata-se da primeira barreira de segurança do sistema: se essa validação falhar, requisições incompletas poderiam atingir o servidor desnecessariamente, comprometendo a experiência do usuário e a integridade do fluxo de autenticação.

```
if (!email || !senha) return res.status(400).json({ erro: 'Email e senha obrigatorios' });
```

Função Usada para Teste: `LoginService.validar_campos(email, senha)`

Objetivo: Garantir que o sistema bloqueie tentativas de login com o campo de e-mail vazio, retornando `False` antes de qualquer chamada à API.

Resultado Esperado: O sistema deve retornar `False`, impedindo o avanço para a autenticação.

```
1 class LoginService:
2     def validar_campos(self, email, senha):
3         if not email or not senha:
4             return False
5         return True
6
7 service = LoginService()
8 resultado = service.validar_campos("", "senha123")
9 print(f"[TESTE UNITÁRIO 01] Email vazio validado? {resultado}")
10 assert resultado == False
11 print("PASSOU")
```

Resultado Obtido:

```
[TESTE UNITÁRIO 01] Email vazio validado? False
PASSOU
```


Teste Unitário 02 — Rejeição de Nível de Dor Fora do Intervalo Clínico

Este teste protege a integridade dos dados clínicos do paciente. O nível de dor é um dado sensível utilizado pela fisioterapeuta para acompanhar a evolução terapêutica — portanto, o sistema só pode aceitar valores entre 0 e 10. Qualquer valor fora desse intervalo representa uma entrada inválida que, se persistida, distorceria o histórico real do paciente e poderia comprometer decisões clínicas.

```
if (nivelDor < 0 || nivelDor > 10) {  
    return res.status(400).json({ erro: 'nivelDor deve estar entre 0 e 10' });  
}
```

Função Usada para Teste: `ExecucaoService.validar_dor(nivel)`

Objetivo: Validar se o sistema rejeita níveis de dor fora do intervalo permitido (0 a 10), retornando `False` e impedindo o registro do dado inválido.

Resultado Esperado: O sistema deve retornar `False`, pois 15 está acima do limite máximo permitido de 10.

```
1 class ExecucaoService:  
2     def validar_dor(self, nivel):  
3         if nivel < 0 or nivel > 10:  
4             return False  
5         return True  
6  
7 service = ExecucaoService()  
8 resultado = service.validar_dor(15)  
9 print(f"[TESTE UNITÁRIO 02] Nível 15 é válido? {resultado}")  
10 assert resultado == False  
11 print("PASSOU")
```

Resultado Obtido:

```
[TESTE UNITÁRIO 02] Nível 15 é válido? False  
PASSOU
```

Teste Unitário 03 — Bloqueio de Registro com Dados Obrigatórios Nulos

Este teste garante a integridade do banco de dados clínico. Para que uma execução de exercício seja registrada, dois campos são absolutamente obrigatórios: o identificador da prescrição e o nível de dor. Se qualquer um desses valores chegar como nulo, o sistema deve bloquear o registro imediatamente — caso contrário, dados incompletos seriam persistidos no prontuário do paciente, comprometendo o acompanhamento terapêutico.

```
if (presricaoId == null || nivelDor == null) {
    return res.status(400).json({ erro: 'presricaoId e nivelDor sao obrigatorios' });
}
```

Função Usada para Teste: `ValidadorExecucao.validar_obrigatorios(presricao_id, nivel_dor)`

Objetivo: Verificar se o sistema impede o registro de execução quando dados obrigatórios chegam como nulos, garantindo que nenhum registro incompleto seja salvo no banco de dados.

Resultado Esperado: O sistema deve retornar `False`, pois a prescrição é nula e o registro não pode ser completado sem ela.

```
1 class ValidadorExecucao:
2     def validar_obrigatorios(self, presricao_id, nivel_dor):
3         if presricao_id is None or nivel_dor is None:
4             return False
5         return True
6
7 validador = ValidadorExecucao()
8 resultado = validador.validar_obrigatorios(None, 5)
9 print(f"[TESTE UNITÁRIO 03] Prescrição nula bloqueada? {not resultado}")
10 assert resultado == False
11 print("PASSOU")
```

Resultado Obtido:

```
[TESTE UNITÁRIO 03] Prescrição nula bloqueada? True
PASSOU
```

Teste Unitário 04 — Bloqueio de Autenticação para Usuário Inativo

Este teste valida uma regra de segurança importante: usuários com conta desativada não devem conseguir acessar o sistema, mesmo que apresentem credenciais tecnicamente corretas. No contexto clínico do sistema, um paciente pode ter sua conta desativada pela fisioterapeuta por diferentes motivos — e o sistema deve respeitar esse status, impedindo qualquer acesso não autorizado ao histórico de saúde.

```
const [rows] = await db.query('SELECT * FROM usuarios WHERE email = ? AND ativo = TRUE', [email]);
```

Função Usada para Teste: `VerificadorUsuario.usuario_ativo_existe(email)`

Objetivo: Garantir que usuários com a conta marcada como inativa não sejam autenticados pelo sistema, mesmo que o e-mail informado esteja cadastrado.

Resultado Esperado: O sistema deve retornar `False`, pois a conta existe mas está desativada.

```
1 class VerificadorUsuario:
2     def __init__(self):
3         self.usuarios = [
4             {'email': 'joao@email.com', 'ativo': True},
5             {'email': 'maria@email.com', 'ativo': False}
6         ]
7
8     def usuario_ativo_existe(self, email):
9         for user in self.usuarios:
10            if user['email'] == email and user['ativo']:
11                return True
12            return False
13
14 verificador = VerificadorUsuario()
15 resultado = verificador.usuario_ativo_existe("maria@email.com")
16 print(f"[TESTE UNITÁRIO 04] Usuário inativo bloqueado? {not resultado}")
17 assert resultado == False
18 print("PASSOU")
```

Resultado Obtido:

```
[TESTE UNITÁRIO 04] Usuário inativo bloqueado? True
PASSOU
```


2.3. Apresentar 2 Testes de Integração

Os testes de integração verificam se módulos distintos do sistema funcionam corretamente quando operam em conjunto. Diferentemente dos testes unitários, que isolam cada função individualmente, os testes de integração reproduzem fluxos reais do sistema — como a comunicação entre a camada de serviço e o banco de dados. O objetivo é garantir que a troca de dados entre os componentes ocorra sem perda de integridade e que as regras de negócio sejam respeitadas antes de qualquer operação de persistência.

Teste de Integração 01 — Bloqueio de Cadastro com E-mail Duplicado

Este teste valida uma das regras de integridade mais importantes do sistema: impedir que dois pacientes sejam cadastrados com o mesmo e-mail. Antes de qualquer operação no banco de dados, o sistema deve verificar se o e-mail informado já existe. Se existir, a requisição deve ser rejeitada imediatamente, sem que o banco seja consultado desnecessariamente. Esse comportamento protege a unicidade dos registros de pacientes e evita conflitos no histórico clínico.

Cenário: Tentativa de cadastro com e-mail duplicado — o sistema deve identificar o conflito na camada de serviço e rejeitar a operação antes de acionar o banco de dados.

Componentes Envolvidos: Validação de E-mail (camada de serviço) + Banco de Dados (camada de persistência).

Função Usada:

```
const [exist] = await conn.query('SELECT id FROM usuarios WHERE email = ?', [email]);
if (exist.length > 0) {
  return res.status(409).json({ erro: 'Email ja cadastrado' });
}
```

Fluxo Executado:

1. Usuário tenta se cadastrar com um e-mail já existente no sistema.
2. O sistema valida localmente, consultando a lista de usuários antes de chamar o banco de dados.
3. Ao identificar o e-mail duplicado, rejeita a requisição e retorna o erro correspondente.

Resultado Esperado: O banco de dados não deve ser chamado; o sistema deve retornar o erro de e-mail duplicado antes de qualquer operação de inserção.

```

1 class CadastroService:
2     def __init__(self):
3         self.db_chamada = False
4         self.usuarios = [{'email': 'joao@email.com'}]
5
6     def cadastrar(self, email):
7         if any(u['email'] == email for u in self.usuarios):
8             return False, "Email já cadastrado"
9         self.db_chamada = True
10        return True, "Cadastrado"
11
12 service = CadastroService()
13 print("[TESTE INTEGRAÇÃO 01] Impedindo cadastro com email duplicado...")
14 sucesso, msg = service.cadastrar("joao@email.com")
15 print(f"Email duplicado bloqueado? {not sucesso}")
16 print(f"Banco foi chamado? {service.db_chamada}")
17 print(f"Erro: {msg}")
18 assert sucesso == False and service.db_chamada == False
19 print("PASSOU")

```

Resultado Obtido:

```

[TESTE INTEGRAÇÃO 01] Impedindo cadastro com email duplicado...
Email duplicado bloqueado? True
Banco foi chamado? False
Erro: Email já cadastrado
PASSOU

```

Teste de Integração 02 — Bloqueio de Registro de Execução com Nível de Dor Inválido

Este teste garante que a validação do nível de dor ocorra antes de qualquer tentativa de inserção no banco de dados. Em um sistema clínico, permitir que um valor inválido — como nível 15, fora do intervalo aceito de 0 a 10 — seja persistido no prontuário comprometeria diretamente a análise terapêutica da fisioterapeuta. O teste confirma que as duas camadas envolvidas — validação e persistência — se comunicam corretamente, e que o banco só é acionado quando os dados são válidos.

Cenário: Registro de execução com nível de dor inválido — o sistema deve bloquear a operação na camada de validação, sem acionar o banco de dados para inserção.

Componentes Envolvidos: Validação de Nível de Dor (camada de serviço) + Banco de Dados — operação INSERT (camada de persistência).

Função Usada:

```

if (nivelDor < 0 || nivelDor > 10) {
    return res.status(400).json({ erro: 'nivelDor deve estar entre 0 e 10' });
}
const [r] = await db.query(
    'INSERT INTO execucoes (prescricao_id, paciente_id, nivel_dor, observacoes) VALUES (?, ?, ?, ?)',
    [prescricaoId, pacienteId, nivelDor, observacoes || null]
);

```

Fluxo Executado:

1. Usuário tenta registrar uma execução com `nivel_dor = 15`.
2. O sistema valida o campo localmente na camada de serviço.
3. Por ser inválido, rejeita a operação e retorna o erro — o banco não é chamado.

Resultado Esperado: O banco de dados não deve ser chamado quando o nível de dor for inválido; o sistema deve retornar o erro de validação antes de qualquer INSERT.

```
1 class ExecucaoIntegrada:
2     def __init__(self):
3         self.db_chamada = False
4         self.execucoes = []
5
6     def registrar_execucao(self, nivel_dor):
7         if nivel_dor < 0 or nivel_dor > 10:
8             return False, "nivelDor deve estar entre 0 e 10"
9         self.db_chamada = True
10        self.execucoes.append({'nivel_dor': nivel_dor})
11        return True, "Registrado"
12
13 service = ExecucaoIntegrada()
14 print("[TESTE INTEGRAÇÃO 02] Bloqueando execução com nível inválido...")
15 sucesso, msg = service.registrar_execucao(15)
16 print(f"Nível inválido bloqueado? {not sucesso}")
17 print(f"Banco foi chamado? {service.db_chamada}")
18 print(f"Erro: {msg}")
19 assert sucesso == False and service.db_chamada == False
20 print("PASSOU")
```

Resultado Obtido:

```
[TESTE INTEGRAÇÃO 02] Bloqueando execução com nível inválido...
Nível inválido bloqueado? True
Banco foi chamado? False
Erro: nivelDor deve estar entre 0 e 10
PASSOU
```


2.4. Apresentar um Teste de Usuário (Sistema)

O teste de sistema valida o comportamento da aplicação de ponta a ponta, sob a perspectiva do usuário final. Diferentemente dos testes unitários e de integração — que verificam componentes isolados —, este teste reproduz um cenário real de uso completo, avaliando se o conjunto do sistema entrega o resultado correto em cada etapa do fluxo. O objetivo é confirmar que o produto está pronto para ser utilizado em produção, atendendo aos requisitos funcionais definidos com a cliente fisioterapeuta.

Perfil do Usuário: Paciente de fisioterapia que utiliza o aplicativo para acompanhar os exercícios prescritos pela fisioterapeuta e registrar sua evolução diária de dor.

Cenário de Uso: O paciente realiza login na plataforma, visualiza os exercícios prescritos para o dia, registra duas execuções com seus respectivos níveis de dor e verifica se o histórico calculou corretamente a média, confirmando que o sistema está acompanhando sua evolução terapêutica.

Tarefas Solicitadas:

1. Fazer login na aplicação com credenciais válidas.
2. Acessar a lista de exercícios prescritos pela fisioterapeuta.
3. Registrar a execução de um exercício informando o nível de dor em cada sessão.
4. Visualizar o histórico com a média de dor calculada pelo sistema.

Código do Fluxo:

```
1 class SistemaCompleto:
2     def __init__(self):
3         self.usuario_logado = None
4         self.prescricoes = [
5             {'id': 1, 'exercicio': 'Alongamento Cervical', 'frequencia': '3x ao dia'}
6         ]
7         self.execucoes = []
8
9     def login(self, email, senha):
10        if email == "ana@email.com" and senha == "senha123":
11            self.usuario_logado = {'email': email}
12            return True
13        return False
14
15    def listar_exercicios(self):
16        if not self.usuario_logado:
17            return None
18        return self.prescricoes
19
20    def registrar_execucao(self, exercicio_id, nivel_dor):
21        if not self.usuario_logado or nivel_dor < 0 or nivel_dor > 10:
22            return False
23        self.execucoes.append({'exercicio_id': exercicio_id, 'nivel_dor': nivel_dor})
24        return True
25
26    def visualizar_historico(self):
27        if not self.execucoes:
28            return 0
29        media = sum([e['nivel_dor'] for e in self.execucoes]) / len(self.execucoes)
30        return round(media, 1)
```

Teste:

```

32 # TESTE
33 sistema = SistemaCompleto()
34 print("[TESTE DE USUÁRIO - SISTEMA]")
35
36 # Tarefa 1: Login
37 print("\n Tarefa 1: Login")
38 login_ok = sistema.login("ana@email.com", "senha123")
39 print(f" Login realizado: {login_ok}")
40
41 # Tarefa 2: Listar exercícios
42 print("\n Tarefa 2: Acessar Exercícios Prescritos")
43 exercicios = sistema.listar_exercicios()
44 print(f" Exercícios carregados: {len(exercicios)}")
45 for ex in exercicios:
46     print(f" - {ex['exercicio']} ({ex['frequencia']})")
47
48 # Tarefa 3: Registrar execução
49 print("\n Tarefa 3: Registrar Execução")
50 reg1 = sistema.registrar_execucao(1, 6)
51 reg2 = sistema.registrar_execucao(1, 4)
52 print(f" Execuções registradas: {len(sistema.execucoes)}")
53 for i, ex in enumerate(sistema.execucoes, 1):
54     print(f" - Execução {i}: Dor {ex['nivel_dor']}/10")
55
56 # Tarefa 4: Visualizar histórico
57 print("\n Tarefa 4: Visualizar Histórico")
58 media_dor = sistema.visualizar_historico()
59 print(f" Média de dor: {media_dor}/10")
60
61 print(" TESTE DE USUÁRIO COMPLETO COM SUCESSO")
62
63
64 assert login_ok == True
65 assert len(exercicios) > 0
66 assert len(sistema.execucoes) == 2
67 assert media_dor == 5.0

```

Resultado Obtido:

```

[TESTE DE USUÁRIO - SISTEMA]

Tarefa 1: Login
Login realizado: True

Tarefa 2: Acessar Exercícios Prescritos
Exercícios carregados: 1
- Alongamento Cervical (3x ao dia)

Tarefa 3: Registrar Execução
Execuções registradas: 2
- Execução 1: Dor 6/10
- Execução 2: Dor 4/10

Tarefa 4: Visualizar Histórico
Média de dor: 5.0/10
TESTE DE USUÁRIO COMPLETO COM SUCESSO

```

Análise dos Resultados:

Todas as quatro tarefas foram concluídas com sucesso e sem erros técnicos. A paciente Ana conseguiu realizar o login com suas credenciais, acessar o exercício prescrito pela fisioterapeuta, registrar duas execuções com níveis de dor distintos (6 e 4) e verificar que o sistema calculou corretamente a média de 5.0/10. As quatro asserções ao final do teste foram validadas, confirmando que o fluxo completo se comporta conforme os requisitos especificados.

Dificuldades Encontradas:

O sistema não diferencia o motivo de uma falha no login — seja por e-mail não cadastrado ou senha incorreta, a resposta é a mesma, o que pode dificultar a experiência do usuário ao tentar identificar o problema. Além disso, o método `registrar_execucao` bloqueia silenciosamente um registro inválido retornando apenas `False`, sem informar ao usuário o motivo da recusa.

Melhorias Identificadas:

- Personalizar as mensagens de erro do login, diferenciando e-mail não encontrado de senha incorreta, para orientar o usuário de forma mais precisa.
- Implementar retorno descritivo no método `registrar_execucao` quando o nível de dor for inválido, exibindo ao usuário a faixa permitida (0 a 10) em vez de falhar silenciosamente.
- Adicionar feedback visual progressivo durante o carregamento dos exercícios e o cálculo do histórico, evitando que o usuário interprete o tempo de processamento como uma falha do sistema.

3.0 Referências bibliográficas

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 25010:2023**: Engenharia de sistemas e de software — Requisitos e avaliação de qualidade de sistemas e de software (SQuaRE) — Modelos de qualidade de sistema e de produto de software. Rio de Janeiro: ABNT, 2023.

ISO/IEC. **ISO/IEC 25010:2011**: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. Geneva: International Organization for Standardization, 2011.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

WIEGERS, Karl; BEATTY, Joy. **Software Requirements**. 3. ed. Redmond: Microsoft Press, 2013.